Analysis

William Hendrix

Lecture 2

Today

- Review
  - Big-Oh notation
- Big-Oh properties
- Analyzing algorithms
- Summation identities
- Recursive analysis

Review: Big-Oh formal definitions

- Provides a useful way to classify functions according to growth rate
  - Focuses on asymptotic (eventual) growth
- O(*g(n)*): grow no faster than *g(n)*
- Ω(*g(n)*): grow no slower than *g(n)*
- Θ(*g(n)*): grow at the same rate as *g(n)*

$f(n) = O(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

$f(n) = \Omega(g(n))$ if and only if there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

$f(n) = \Theta(g(n))$ if and only if there exist positive constants c_1, c_2, and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

- *To prove:* find positive constants that satisfy definition

Proof advice

- General proof strategy: find values of c and n_o
  - "Fill-in-the-blank" proof

$$f(n) \leq (\underset{\text{pos. const.}}{\underline{\qquad\qquad}})g(n) \quad \forall n \geq (\underset{\text{pos. const.}}{\underline{\qquad\qquad}})$$

- Finding smallest c or n_o isn't necessary
  - Choosing well can make your life easier, though
- Small powers of n are bounded above by larger powers
- Functions with negative coefficients are bounded above by zero
- Large coefficients are bounded above by n
  - With sufficiently large n_o

Big-Oh example (series)

- Prove that $\sum_{i=1}^{n} i = \Omega(n^2)$.

Connection to calculus

- You can also determine O, Ω, and Θ by limits:

g grows faster $\longrightarrow$ $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \quad \to f(n) = O(g(n))$

Actually $f(n) = o(g(n))$

Same growth rate $\longrightarrow$ $\lim_{n\to\infty} \frac{f(n)}{g(n)} \in (0, \infty) \to f(n) = \Theta(g(n))$

g grows slower $\longrightarrow$ $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty \quad \to f(n) = \Omega(g(n))$

Actually $f(n) = \omega(g(n))$

- Standard rules for taking limits apply
  - Including L'Hôpital's Rule

Analysis of Big-Oh

Pros

- Provides a useful summary of the growth rate of the complexity
- Compact
- Simple: eight classes cover most useful algorithms

$$O(1) \ll O(\lg n) \ll O(n) \ll O(n \lg n) \ll O(n^2) \ll O(n^3) \ll O(2^n) \ll O(n!)$$

Cons

- Ignores contributions from coefficients and lower-order terms
- Doesn't rank algorithms with same growth rate
- Doesn't rank algorithms on small inputs
- Some of the "best" algorithms have extremely large coefficients, making them impractical for many purposes

Formal definition extra practice

- Use the *formal definition* of Big-Theta to prove:

  For any $x > 0$, if $f(n) = \Theta(g(n))$, then $xf(n) = \Theta(g(n))$

Properties of Big-Oh notation

- Reflexivity

  $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$, and $f(n) = \Theta(f(n))$

- Anti-symmetry

  $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$

  $f(n) = O(g(n))$ and $f(n) = \Omega(g(n)) \Leftrightarrow f(n) = \Theta(g(n))$

- Symmetry (Θ only)

  $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$

- Transitivity

  $f(n) = O(g(n))$ and $g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$

  $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \rightarrow f(n) = \Omega(h(n))$

  $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \rightarrow f(n) = \Theta(h(n))$

- Alternately:

  $O(O(h(n))) = O(h(n))$

  $\Omega(\Omega(h(n))) = \Omega(h(n))$

  $\Theta(\Theta(h(n))) = \Theta(h(n))$

Combination properties

- Envelopment
  - Addition

    $$O(f(n)) + O(g(n)) = O(f(n) + g(n))$$
    $$\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n) + g(n))$$
    $$\Theta(f(n)) + \Theta(g(n)) = \Theta(f(n) + g(n))$$
  - Multiplication

    $$O(f(n))O(g(n)) = O(f(n)g(n))$$
    $$\Omega(f(n))\Omega(g(n)) = \Omega(f(n)g(n))$$
    $$\Theta(f(n))\Theta(g(n)) = \Theta(f(n)g(n))$$
- All three ignore constant coefficients

$$\forall x > 0, \quad \begin{aligned} f(n) = O(g(n)) &\rightarrow xf(n) = O(g(n)) \\ f(n) = \Omega(g(n)) &\rightarrow xf(n) = \Omega(g(n)) \\ f(n) = \Theta(g(n)) &\rightarrow xf(n) = \Theta(g(n)) \end{aligned}$$

- Only the largest term matters

$$f(n) = O(g(n)) \rightarrow O(f(n) + g(n)) = O(g(n))$$
$$f(n) = O(g(n)) \rightarrow \Omega(f(n) + g(n)) = \Omega(g(n))$$
$$f(n) = O(g(n)) \rightarrow \Theta(f(n) + g(n)) = \Theta(g(n))$$

Big-Oh properties example

- Use Big-Oh properties to establish the following:

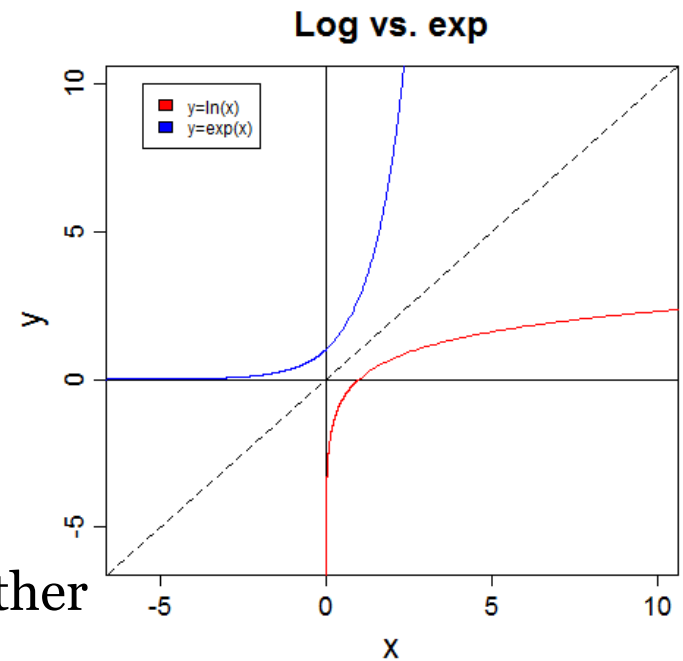
1. If $f(n) = 13n^2 + 1234n + 91.2n\sqrt{n}$, then $f(n) = \Theta(n^2)$. Use the facts that $n = O(n^2)$ and $\sqrt{n} = O(n)$.

Revenge of the logarithms

- **Logarithm:** inverse exponential function

$$y = \ln x \Leftrightarrow x = e^y$$

- Natural log (ln): inverse of e^x
- Logarithms of other bases: $\log_b(x)$
  - $\log_2(x)$ is very common in algorithms
- Computing logs of other bases
  - $\log_b(x) = \frac{\ln x}{\ln b}$
  - All logs are *scalar multiples* of one another



- **Log properties**

	Because
Base 2 → $\lg(ab) = \lg(a) + \lg(b)$	$2^A 2^B = 2^{A+B}$
$\lg(a^b) = b\lg(a)$	$\left(2^A\right)^b = 2^{Ab}$
$\sum_{i=1}^{n} \frac{1}{i} = \Theta(\lg n)$	$\int_1^n \frac{1}{x} dx = \ln n$

Logarithm property example

- Prove that $\lg(n!) = \Theta(n \lg(n))$.

Back to our previous example…

Input: $data$: array of integers
Input: n: size of $data$
Output: index min such that $data[min] \leq data[j]$, for all j from 1 to n

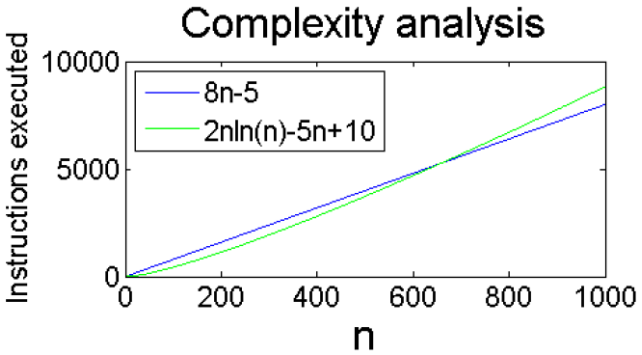
```
Algorithm: FindMin
min = 1
for i = 2 to n do
    if data[i] < data[min] then
        min = i
    end
end
return min
```

Ops per line	Times executed
…	…

Total ops: $\leq 7(n-1) + 2 = 7n - 5$

Question: is this better or worse than an algorithm that takes at most $2n \ln n - 5n + 10$ ops?

Better unless n < 395



Worst case: *O(n)*

Don't need to count instructions!

Other algorithm: *O(n*lg*(n))*

Conclusion: Our algorithm is better for sufficiently large *n*.

Algorithm analysis

- Identify loops and function calls
  - Everything else is $\Theta(1)$
- *For loops*:
  - Estimate number of iterations
  - Estimate loop body running time
  - Does loop run time depend on iteration #?
    - If not: total = # iterations * time per iteration
    - Otherwise: total = sum of all iterations
- *For functions:*
  - Non-recursive: analyze separately
  - Recursive: set up a recurrence and solve (after exam)
- Overall complexity: sum of complexities
  - Largest loop/function call

Summations

- Summations show up frequently in analysis

- Notation
  - "Dotty notation": $1 + 2 + 3 + \ldots + n$
    - Easy to understand
  - Compact notation: $\sum_{i=1}^{n} i$
    - Easy to write

- Useful identities

$$\sum_{i=1}^{x} i = \Theta(x^2) \qquad \sum_{i=1}^{x} 2^i = \Theta(2^x)$$

$$\sum_{i=1}^{x} \frac{1}{i} = \Theta(\lg x) \qquad \sum_{i=1}^{x} \frac{1}{2^i} = \Theta(1)$$

Loop analysis example

- What is the worst-case time complexity for the following algorithm?
  - What is the best-case?

```
Input: data: array of integers
Input: n: length of data
Output: permutation of data such that
        data[1] ≤ data[2] ≤ ... ≤ data[n]
Algorithm: SelectionSort
for i = 1 to n − 1 do
    min = i
    for j = i + 1 to n do
        if data[j] < data[min] then
            min = j
        end
    end
    Swap data[i] and data[min]
end
return data
```

Analysis further practice

1. What is the *worst-case* time complexity for the algorithm below? Show your work.
   - *Best case?*

Input: $data$: an array of integers to sort
Input: n: the number of values in $data$
Output: permutation of $data$ such that
$data[1] \leq \ldots \leq data[n]$

```
Algorithm: InsertionSort
for i = 2 to n do
    ins = data[i]
    j = i
    while j > 1 and data[j − 1] > ins do
        data[j] = data[j − 1]
        j = j − 1
    end
    data[j] = ins
end
return data
```

Tricky example

- Algorithm analysis is **not** simply counting the number of loops

```
Input: data: array of integers
Input: n: length of data
Output: permutation of data such that
        data[1] ≤ data[2] ≤ ... ≤ data[n]
Algorithm: BadSort
for i = n − 1 to 1 step −1 do
    for j = 1 to n − i step i do
        if data[j] > data[j + i] then
            Swap data[j] and data[j + i]
        end
    end
end
return data
```

Coming up

- Recursive analysis
- Master Theorem
- Data structures review

- **Recommended reading (today):** Section 11.1
  - *Practice problems:* R-11.1, C-11.1, C-11.3 (complexity only)
- **Recommended reading:** Sections 2.2, 1.4, and 2.1
  - *Practice problems:* R-2.3, R-2.2, R-2.6abc, C-2.12, C-2.19, A-2.1